

2026-06-10-D-cli-bridge

Workstream D — CLI-agent bridge as model providers

Date: 2026-06-10 **Phase:** PLANNING (read-only analysis) **Author:** analysis subagent **Scope:** Make each working CLI coding agent (Claude Code, Qwen Code, OpenCode, Gemini CLI, Crush, aider, codex, plandex, ...) its OWN HelixCode model-provider that proxies prompts via the agent's CLI and surfaces power-features (Vision, Generative, RAG, Memory, MCP, LSP, ACP). Primary path = system-installed binary; fallback = locally-built submodule.

Evidence rule: every claim below cites a real `file:line` or real command output captured read-only this session. Items not found are marked **ABSENT**. No code was modified.

1. Inventory of `cli_agents/*` submodules

`ls cli_agents/` returned **51 entries** (all are git submodules per `.gitmodules`; cross-checked `.gitmodules:64-164+`). Classified by whether they are actual CLI **coding agents** (bridgeable) vs tooling/MCP/infra:

Actual CLI coding agents (bridge candidates): `aider`, `claude-code`, `claude-code-source`, `codex`, `codex-skills`, `crush`, `forge`, `gemini-cli`, `qwen-code`, `plandex`, `codename-goose`, `cline`, `aiagent`, `aichat`, `aichat-llm-functions`, `amazon-q`, `copilot-cli`, `deepseek-cli`, `deepseek-cli-youkpan`, `mistral-code`, `gpt-engineer`, `gptme`, `junie`, `nanocoder`, `octogen`, `open-interpreter`, `shai`, `swe-agent`, `vtcode`, `warp`, `xela-cli`, `zeroshot`, `codai`, `get-shit-done`, `bridle`, `snow-cli`, `multiagent-coding`, `taskweaver`, `ui-ux-pro-max`, `agent-deck`, `noi`, `superset`, `x-cmd`.

Orchestration / harness / not a direct coding-CLI: `claude-squad` (multiplexer), `cli-agent`, `conduit`, `spec-kit` (workflow), `git-mcp`, `postgres-mcp` (MCP servers, not agents), `fauxpilot` (server).

Resources (NOT agents) — `cli_agents_resources/` (`.gitmodules:16-32`): `Awesome-AI-Agents`, `Awesome-AI-GPTs`, `Cheshire-Cat-Ai`, `GitHub-Awesome-Copilot`, `OpenAI-Cookbook`, `Taches-CC-Resources`. Documentation/cookbooks only — exclude from bridging.

2. System-installed detection (read-only command `-v + --version`)

Captured this session (command `-v <tool> + cheap <tool> --version`):

Agent	On PATH	Path	Version
claude (Claude Code)	✓	/Users/ milosvasic/.local/ bin/claude	2.1.170 (Claude Code)
qwen (Qwen Code)	✓	/opt/homebrew/bin/ qwen	0.5.0
opencode	✓	/Users/ milosvasic/.opencode/ bin/opencode	1.16.2
gemini (Gemini CLI)	✓	/opt/homebrew/bin/ gemini	0.1.9
crush	✓	/opt/homebrew/bin/ crush	v0.22.2
codex	✓	/opt/homebrew/bin/ codex	(no output to -- version; binary present)
goose	✓	/Users/ milosvasic/.local/ bin/goose	1.8.0
copilot (GitHub Copilot CLI)	✓	/Users/ milosvasic/.local/ bin/copilot	GitHub Copilot CLI 1.0.46.
aider	✗ ABSENT	—	(fallback to submodule build)
plandex	✗ ABSENT	—	(fallback to submodule build)
forge	✗ ABSENT	—	(fallback to submodule build)
q / amazon-q	✗ ABSENT	—	—
cursor / cody	✗ ABSENT	—	—
continue	⚠ FALSE POSITIVE	—	command -v continue matched the shell builtin continue, not a binary. Treat as ABSENT.

8 agents are bridgeable via the PRIMARY (system-installed) path today: claude, qwen, opencode, gemini, crush, codex, goose, copilot. aider/plandex/forge require the FALLBACK (submodule-build) path.

3. Existing bridge code

3.1 helix_code/internal/llm — HTTP/API providers only, NO CLI-exec bridge

- The Provider interface lives at `helix_code/internal/llm/missing_types.go:356` — methods: `GetType`, `GetName`, `GetModels`, `GetCapabilities`, `Generate`, `GenerateStream`, `IsAvailable`, `GetHealth`, `Close`, `GetContextWindow`, `CountTokens`. This is the contract a per-CLI-agent provider must satisfy.
- `helix_code/internal/llm/copilot_provider.go` is **NOT a CLI bridge** — it is HTTP/token-based: it loads a GitHub token (`loadGitHubCLIToken : 108`), exchanges it (`exchangeGitHubToken : 151`), and makes OpenAI-style HTTP requests (`makeOpenAIRequest : 447`). No `exec.Command` of the `copilot` binary.
- `grep "exec.Command|os/exec" helix_code/internal/llm/*.go` hits only `auto_llm_manager.go`, `local_llm_manager.go`, `model_converter.go`, `model_download_manager.go` — all for local-model/process management, **not** CLI-agent proxying.
- `grep "opencode|qwen-code|claude-code|cli_agent" over helix_code/internal/llm/*.go` → **empty**. **There is no CLI-agent provider in helix_code/internal/llm today.**

3.2 submodules/helix_agent/internal/clis — the REAL CLI-exec orchestration layer (separate Go module `dev.helix.agent`)

This is where CLI bridging actually exists: - `agents/registry.go:107`

`AgentInfo{Type, Name, Description, Vendor, Version, Capabilities, IsEnabled, Pri`

`AgentIntegration` interface: `Info / Initialize / Start / Stop /`

`Execute(ctx, command, params) / Health / IsAvailable`. - `agents/`

`base/base.go:120` `IsAvailable()` does a **real**

`exec.LookPath(string(b.info.Type)); :127` `ExecuteCommand` runs

`exec.CommandContext`. This is the genuine binary-detect + `exec` primitive. - **~70 per-agent**

packages under `agents/` (incl. `claude_code`, `qwencode`, `gemini`, `crush`, `aider`,

`codex`, `opencodecli`, `plandex`, `copilotcli`, `codenamegoose`, `forge`, `cline`,

`openhands`, `continueagent`, `kiro`, ...) — see `ls agents/. - clis/`

`instance_manager.go` (38 KB) is a pooled lifecycle manager: `CreateInstance: 83`,

`AcquireInstance: 167`, `SendRequest: 290`, `performHealthCheck: 852`, plus per-

agent `executeAider: 906`, `executeClaudeCode: 915`, `executeCodex: 924`, ... (one

dispatch method per agent type). Backed by `pool.go`, `event_bus.go`, SQLite persistence

(`persistInstance: 517`). - The richer `CLIAgent` interface (`clis/types.go:537`)

adds `ExecuteTask/GetCapabilities`; `Capabilities` struct (:555) =

`SupportsRepoMap/GitOps/DiffEditing/ToolUse/Browser/Sandbox/`

`Interpreter/SupportedTools`; `DefaultCapabilities(:567)` hardcodes per-

agent caps (e.g. `ClaudeCode` tools `Bash/Read/Write/Edit/Glob/Grep`).

3.3 🚩 Two anti-bluff / decoupling problems found

1. **Many concrete agents are STUBS, not real exec bridges.** `agents/qwencode/qwencode.go` `generate()` (:101) returns the hardcoded string `// Generated by Qwen\n// %s"; chat()` (:115) returns `fmt.Sprintf("Qwen: %s", message)`; `IsAvailable()` (:137) checks `config.APIKey != ""` — it never execs the `qwen` binary. This is a BLUFF surface (§Rule 2 / §11.4) that Workstream D must

- replace with a real exec to the installed qwen CLI. Each of the ~70 agents must be audited individually — `base.go` is honest, but per-agent overrides may not be. (`claude_code/claude_code.go` is more real: `Start:151` checks `exec.LookPath("node")`, `handleBash:276 / handleGit:310` actually exec.)
2. `helix_code` **does NOT consume** `dev.helix.agent.grep "helix.agent"`
`helix_code/go.mod` → NONE. The `helix_code/go.mod` replace block (:194-206) wires `containers/helixqa/docprocessor/llm_orchestrator/vision_engine/challenges/security` but **not** `dev.helix.agent`. So the CLI-exec layer is in a module HelixCode's LLM provider registry can't currently import. Workstream D must either (a) add a replace `dev.helix.agent => ../submodules/helix_agent + adapter`, or (b) port the bridge into `helix_code/internal/llm` as `cli_agent_provider.go` (CONST-051 decoupling favors keeping the generic bridge in the submodule and writing a thin HelixCode adapter).
-

4. Existing CLI configs (`cli_agents_configs/`) — direction is REVERSED from Workstream D

- `cli_agents_configs/` holds **109 files** (`ls | wc -l`) — `<agent>.json + <agent>.yaml` pairs for ~50 agents, plus `README.md`.
- `README.md`: 3 states these are “exported configuration files for all 47 CLI agents that integrate with HelixAgent.”
- **Shape** (`claude-code.json:1-40`, **identical in** `qwen-code.json:1-30`): each config points the CLI agent **AT HelixAgent** as an OpenAI-compatible backend:

```
"provider": { "type": "openai-compatible", "name":
              "helixagent",
              "base_url": "http://localhost:7061/v1",
              "api_key_env": "HELIXAGENT_API_KEY" },
"models": [{ "id": "helixagent-debate", "capabilities":
              ["vision","streaming","function_calls","embeddings","mcp","acp","lsp"]
"mcp": { "helixagent-acp": {"url":".../v1/acp"}, "helixagent-
          cognee": {...},
          "helixagent-embeddings": {...}, "helixagent-lsp":
          {...} }
```

⚠ This is the **opposite** direction of Workstream D: today's configs make *CLI agent* → *HelixAgent*. Workstream D wants *HelixCode* → *CLI agent*. The two can coexist (a CLI agent can both serve HelixCode AND be backed by HelixAgent), but the re-export loop here is for the existing direction.

4.1 The re-export/install/validate machinery ALREADY EXISTS (in `helix_agent`)

- Generator entry point: `submodules/helix_agent/cmd/helixagent/main.go:107` flag `--generate-all-agents`, :108 `--all-agents-output-dir`; handler `handleGenerateAllAgents:4522` loops agents and `os.WriteFile(outputPath, jsonData, 0644)` (:4594). Per-agent handlers

exist too: `handleGenerateOpenCode:2378`, `handleGenerateCrush:3734`, `handleGenerateAgentConfig:4348`.

- A **unified generator + validator** lives at `submodules/helix_agent/challenges/codebase/go_files/unified_cli_generator/unified_cli_generator.go`: `GenerateAll():76`, `Generate(agentType):97`, `saveConfig():212` (`os.WriteFile:244`), and crucially `Validate(agentType, config):263` returning a `ValidationResult{Valid}` — this is the **validate** leg of the re-export+install+validate loop. Per-agent generators also exist (`crush_generator/`, `opencode_generator/`, `kilocode_generator/`).
 - **README quotes the operator workflow** (`README.md`): `./bin/helixagent --generate-all-agents --all-agents-output-dir=./cli_agents_configs`.
 - **ABSENT**: an *install* step that copies a generated config to each agent's real config location (e.g. `~/ .claude.json`, `~/ .qwen/`, `~/ .config/opencode/`, `~/ .config/crush/`) and then re-runs the agent to validate. Generation + in-memory validation exist; **filesystem install + post-install live validation is the gap**. Also ABSENT: any config that re-exports `HelixCode` as a *consumer-of-CLI-agent* (the Workstream-D direction).
-

5. Design — per-CLI-agent provider contract (sketch)

5.1 New provider in `helix_code/internal/llm` (one per CLI agent, or one parametrized `CLI-Agent-Provider`)

Implement `llm.Provider` (`missing_types.go:356`). Recommended: a single `CLI-Agent-Provider` struct parametrized by an `AgentSpec` (so adding an agent = a registry row, not a new file), satisfying: - `GetType()` → new `ProviderType` per agent (extend `missing_types.go:39-46` set, e.g. `ProviderTypeClaudeCodeCLI`, `ProviderTypeQwenCodeCLI`, `ProviderTypeOpenCodeCLI...`). - `GetName()` → agent display name. - `IsAvailable(ctx)` → `real exec.LookPath(primaryBinary)`; if absent, resolve FALLBACK submodule build path (see 5.3). `Mirror base.go:120`, **never** the `qwencode APIKey != ""` bluff (§3.3). - `Generate(ctx, req)` → `exec.CommandContext(ctx, binary, nonInteractiveArgs...)`, write prompt to stdin or `--prompt`, capture stdout, parse, surface real exit code. Pattern source: `base.go:127 + claude_code.go:276/310`. - `GenerateStream` → stream stdout line-buffered into the channel. - `GetModels()` → DYNAMIC per agent (CONST-036, no hardcoded lists): exec the agent's model-list subcommand and parse (TODO-research exact flags per agent — see 5.4). - `GetCapabilities()` → map the agent's `clis.Capabilities` (`types.go:555`) into power-feature flags (Vision/Generative/RAG/Memory/MCP/LSP/ACP). Per CONST-040 these SHOULD ultimately come from `LLMsVerifier VerificationResult`, not hardcoded `DefaultCapabilities` (`types.go:567`). - `GetHealth` → reuse `clis.performHealthCheck` semantics; `CountTokens/GetContextWindow` → char-fallback like `copilot_provider.go:312`.

5.2 Reuse vs reimplement (CONST-051 / §11.4.74)

Prefer **adapting the existing** `submodules/helix_agent/internal/clis` **layer** (real `exec` + pool + lifecycle + registry of ~70 agents) over writing fresh `exec` in `helix_code`. Action: add `replace dev.helix.agent => ./submodules/helix_agent` to `helix_code/go.mod` and write a thin `cli_agent_provider.go` adapter that wraps `clis.InstanceManager / agents.AgentIntegration` behind `llm.Provider`.

BUT first audit each agent package for stubs (§3.3 item 1) and replace bluff bodies with real `exec` to the installed CLI.

5.3 Primary-vs-fallback selection

1. **PRIMARY:** `exec.LookPath(<agentBinary>)` → if found, use system-installed (the 8 in §2).
2. **FALLBACK:** if absent, resolve the submodule-built binary under `cli_agents/<agent>/` (e.g. built artifact in the submodule's `bin//dist//target/`, or a documented build target). `aider/plandex/forge` will need this today.
3. Record which path was chosen in `GetHealth` details (anti-bluff evidence).

5.4 Re-export + install + validate loop (Workstream-D direction)

1. **Re-export:** extend the helix-agent generator (`unified_cli_generator.go`) — or add a HelixCode-side exporter — to emit each agent's config (generation already proven, :76/:212).
2. **Install:** NEW — copy the generated config into the agent's real config dir, backing up/ replacing existing (this is the ABSENT step in §4.1).
3. **Validate:** call `UnifiedCLIGenerator.Validate(:263)` for static validation, THEN run the installed agent end-to-end (real `exec`, captured output) to prove the install works (§11.4.5 runtime evidence).

5.5 TODO-research items (require official-docs verification per §11.4.99)

- **OpenCode incl. Zen provider models:** exact non-interactive invocation (`opencode run?`), model-list command, and how OpenCode “Zen” provider models are enumerated/ selected. (`opencode 1.16.2` present.)
- **Claude Code:** non-interactive/print mode flags (`claude -p/--print/--output-format json?`), model selection, MCP wiring, session reuse — confirm against Claude Code 2.1.170 docs.
- **Qwen Code:** real CLI invocation + model list (current `qwencode.go` is a stub — must be rebuilt against `qwen 0.5.0`).
- **Gemini CLI / Crush / codex / goose / copilot:** per-agent non-interactive flags, JSON output mode, and dynamic model-list subcommands.
- Power-feature surfacing (Vision/RAG/Memory/MCP/LSP/ACP) per agent — which agents expose which, and how to probe them at runtime for CONST-040 compliance.

Executive summary (12 lines)

1. **System-installed NOW (PRIMARY path ready, 8):** `claude 2.1.170`, `qwen 0.5.0`, `opencode 1.16.2`, `gemini 0.1.9`, `crush v0.22.2`, `codex`, `goose 1.8.0`, `copilot 1.0.46`.
2. **ABSENT (need FALLBACK submodule build):** `aider`, `plandex`, `forge`, `amazon-q/q`, `cursor`, `cody`.
3. `continue` was a FALSE POSITIVE (shell builtin, not a binary) — treat as absent.
4. `cli_agents/` has 51 agent submodules (~43 are real coding-CLI bridge candidates); `cli_agents_resources/` 6 entries are docs only.
5. **No CLI-exec provider exists in `helix_code/internal/llm` —** `copilot_provider.go` is HTTP/token-based, not a CLI bridge; the `Provider` contract is `missing_types.go:356`.

6. **The real CLI-exec layer is** `submodules/helix_agent/internal/clis` (module `dev.helix.agent`): `AgentIntegration` registry, `base.go` real `exec.LookPath+exec.CommandContext`, ~70 per-agent pkgs, pooled `instance_manager.go`.
7. ⚠ **Many per-agent packages are STUBS** — `qwencode.go` returns hardcoded strings and gates on `APIKey!=""`, never exec'ing the CLI (a §11.4 bluff to fix).
8. ⚠ `helix_code` **does not depend on** `dev.helix.agent` (`helix_code/go.mod` has no such replace) — a wiring gap to close.
9. `cli_agents_configs/ (109 files)` **points each CLI agent AT HelixAgent** (openai-compatible `localhost:7061/v1`) — the REVERSE of Workstream D's direction.
10. **Re-export + static-validate machinery already exists** in `helix_agent` (`--generate-all-agents main.go:4522`; `unified_cli_generator.go` `GenerateAll/saveConfig/Validate`); **filesystem INSTALL + live post-install validation is ABSENT**.
11. **Provider-contract sketch:** one parametrized `CLIAgentProvider` implementing `llm.Provider`, real `exec.LookPath`→primary / submodule→fallback, dynamic `GetModels` via agent subcommand (CONST-036), capabilities from `LLMsVerifier` (CONST-040), reusing the `helix_agent clis` layer via a thin adapter (CONST-051/ §11.4.74).
12. **Next steps:** audit & de-bluff each agent pkg; add `replace dev.helix.agent`; build the `HelixCode`→CLI provider + install/validate loop; research per-agent non-interactive flags (OpenCode Zen, Claude Code -p, Qwen) per §11.4.99.